

AI workflows 的分层调度：解析基于DeepSeek Pro/Flash混合模型的智能管道与数据架构

核心处理流程：三阶段脚本生成管道的机制与协同

项目的智能核心在于其高效且高质量的脚本生成能力，这主要通过一个精心设计的“三阶段脚本生成管道”来实现。该管道借鉴了现代软件开发和人工智能代理领域的最佳实践，将复杂的编码任务分解为一系列更小、更专注、可管理的子任务^{17 21}。这种分阶段的方法论旨在通过专业化分工来提升最终产出的质量与可靠性，并为后续的调试、追溯和版本控制奠定坚实的基础。整个流程可以被解构为三个紧密衔接的阶段：意图理解与规划、代码生成与初步验证、以及优化与重构。

第一阶段的核心任务是“意图理解与规划”。在此阶段，系统接收用户的初始需求或自然语言描述作为输入¹⁹。这些需求通常是模糊、非结构化的，需要被转化为一份明确、精确且机器可读的指令集。为了完成这一高阶认知任务，系统会调用一个具有强大推理能力的大型语言模型，即DeepSeek V4 Pro^{57 58}。V4 Pro凭借其卓越的长上下文理解和世界知识能力，能够深入剖析用户的需求，识别其中的关键约束和隐含意图⁵⁷。其输出并非直接的代码，而是一份结构化的中间产物，例如遵循特定模式的YAML格式文件或伪代码计划²⁰。这个规划蓝图是整个生成过程的“源”，其准确性至关重要。它不仅定义了代码的功能模块划分，还可能包含函数签名、数据结构定义以及高层次的算法流程。为了增强规划的质量，系统可能会利用V4 Pro的“思维链”（Chain-of-Thought）功能，让模型先输出其思考过程，再给出最终规划，从而提高规划的合理性和鲁棒性³⁷。此外，对于特别复杂的请求，系统还可以设置更高的计算资源投入（reasoning effort），以换取更高质量的初始规划³⁹。这个阶段的成功与否，直接决定了后续所有工作的质量上限。

第二阶段聚焦于“代码生成与初步验证”。该阶段的输入是第一阶段生成的结构化规划。执行者依然是一个强大的LLM，但根据规划的性质，也可能动态切换到其他更合适的模型。考虑到代码生成是一项高度模式化的任务，同时又需要快速迭代和低成本，DeepSeek V4 Flash成为此阶段的理想选择^{8 59}。V4 Flash以其极高的推理速度和远低于Pro版的成本而著称，使其在需要大量试错和快速响应的场景下具备显著优势^{8 9}。此阶段的目标是根据第一阶段的规划，生成符合规范的、可运行的代码片段或完整的脚本。一种高效的实践范式是在此阶段引入单元测试的生成。参考Claude Code的工作流，系统可以先根据规划编写针对每个模块的单元测试，然后再实现对应的业务逻辑¹⁷。这种方式确保了代码从一开始就被置于严格的验证框架之下，大大降低了后期修复缺陷的成本。初步验证的结

果，无论是通过静态代码分析工具还是自动运行的单元测试，都会被反馈给系统。如果验证失败，流程将进入一个重要的闭环环节，而不是简单地终止。系统会分析失败原因，并尝试调整代码生成的提示词或参数，或者触发重新生成，从而实现自我修正。

第三阶段是“优化与重构”。此阶段的输入是第二阶段成功生成并验证过的代码及其相关的测试结果。尽管代码已经能够正常工作，但它可能在性能、可读性、可维护性或代码风格上仍有改进空间。因此，此阶段的目标是对现有代码进行深度打磨，产出高质量的最终版本。鉴于优化和重构往往需要对整个代码块有宏观的审视能力，并进行创造性修改，回归使用像DeepSeek V4 Pro这样的高性能模型是合理的选择 [58](#)。V4 Pro的强大推理能力有助于发现潜在的性能瓶颈、冗余逻辑，并提出更优雅的解决方案。同样，此阶段也可以再次采用“先写测试后重构”的模式，确保在修改过程中不会引入新的错误。经过此阶段的精炼，最终交付给用户的脚本不仅功能正确，而且结构清晰、易于维护，充分体现了工业级代码的标准。整个三阶段管道的设计，不仅是一个线性的任务执行流程，更是一个包含反馈和迭代的闭环系统。它模拟了人类专家开发者的工作方式：先制定详尽计划，然后动手编码并不断测试，最后进行代码审查和优化。这种结构化的处理流程，是保障项目在面对复杂、多变的用户需求时，依然能稳定产出高质量成果的关键所在。

数据模型设计：支撑双向追溯与历史管理的基石

为了支撑流畅的用户交互，特别是撤销/重做历史功能和跨阶段的双向追溯，系统必须构建一个健壮、高效且灵活的数据模型。仅依赖单一的数据存储机制无法同时满足低延迟的实时编辑、可靠的长期状态管理和精确的历史追踪需求。因此，业界的最佳实践是采用一种组合式的存储策略，巧妙地结合了“完整状态快照”和“增量变更记录”两种技术。这种混合模型不仅解决了技术上的挑战，也为实现卓越的用户体验奠定了基础。

在该数据模型中，**YAML**快照扮演着应用“权威状态”副本的角色。每当用户完成一个有意义的操作节点，例如成功完成一个阶段的脚本生成、手动保存当前工作区，或者系统自动进行了一次重要更新后，系统便会捕获并持久化一份完整的YAML对象作为快照。这种方法的主要优势在于其恢复状态的简洁性和高效性。当用户需要返回到某个特定的时间点时，系统可以直接加载对应的快照文件，整个应用的状态便能瞬间复原，保证了状态的绝对完整性和一致性。这对于实现“重做”功能至关重要，因为“重做”本质上就是从旧的快照状态出发，重新执行一系列操作。然而，YAML快照的劣势也十分明显，即存储空间的消耗巨大。对于结构复杂、内容庞大的YAML文档，频繁地创建和存储快照会迅速累积起可观的存储成本。

与之相对的是**JSON Patch**，它用于记录状态的“增量变更”。每一次微小的用户编辑操作，例如修改一个字段的值、在列表中添加或删除一个元素，都会被严格地表示为一个符合

RFC 6902标准的JSON Patch操作对象 1。这种基于差异的记录方式具有无与伦比的存储效率，因为它只记录变化的部分，与状态的总大小无关。这使得它完美适用于处理高频次、细粒度的实时编辑。更重要的是，JSON Patch天然支持撤销操作。每一条正向的Patch都可以计算出其对应的逆向Patch，从而实现对单步操作的精确回滚 1。这为用户提供了一个即时、平滑的撤销体验。但JSON Patch的劣势在于，若要恢复到一个较早的历史状态，系统需要重新应用或反向应用一长串补丁序列，这会带来较大的计算开销，尤其是在历史记录非常久远的情况下。

一个极具启发性的设计范例来自名为PatchBoard的架构，它主张在多个AI代理之间，通过交换经过验证的JSON Patch来协作修改一个共享的、由Schema定义的JSON对象 6 41。这一理念完全可以无缝迁移到本项目中。应用的核心配置或生成的脚本逻辑可以被建模为一个遵循特定JSON Schema的YAML对象。上述三阶段生成管道中的每一个阶段，都可以被视为一个独立的“Agent”或“Service”。它们不直接修改原始对象，而是专注于生成一系列合法的、符合Schema约束的JSON Patch。系统在将这些Patch应用于主对象之前，可以增加一个验证步骤，确保所有变更都是安全和有效的。这种基于Patch的协作方式极大地增强了系统的鲁棒性，避免了因非法操作导致的数据损坏，并使得整个修改过程变得透明、可审计。

这种混合数据模型的设计对用户体验产生了深远的影响。首先，由于绝大多数实时编辑都以JSON Patch的形式被快速记录和应用，用户可以获得近乎即时的反馈，感觉系统反应灵敏。同时，他们可以轻松地撤销最近的几次操作，这种“无限撤销”的自由感极大地鼓励了探索和实验。其次，定期的YAML快照确保了即使在发生严重错误或意外情况后，用户也能通过加载最近的快照回到一个已知的良好状态，提供了强大的安全保障。最后，也是最精妙的一点，是实现了无缝的双向追溯。当一个由AI生成的代码块（例如，在第二阶段生成的一个函数）被创建时，系统可以在该代码块的元数据中嵌入一个source_ref字段。这个字段可以指向第一阶段生成的、对该函数有直接贡献的规划节点。反之，当用户在界面上点击UI中的某个规划项时，系统可以高亮显示所有由它衍生出的、在下游阶段生成的代码块。这种链接关系使得用户能够在“意图-规划-实现”的层次间自由穿梭，极大地提升了对生成结果的理解、调试和修改效率，完全契合了SRS文档中关于行为审查和关系图谱的要求 47。

特性	YAML 快照	JSON Patch
记录内容	应用在某一时间点的完整状态	状态的增量式变更
优势	恢复状态简单高效，保证完整性；易于实现重做功能	存储效率极高，适合高频次微小编辑；天然支持撤销
劣势	存储空间消耗大，尤其适用于大型对象	恢复早期状态计算开销大；需要维护一个补丁序列
适用场景	手动/自动保存点；阶段结束标志；历史里程碑	实时键盘输入、鼠标拖拽等细粒度用户交互

撤销/重做历史功能的数据建模与用户体验

撤销/重做功能是现代交互式软件不可或缺的核心特性，它赋予用户犯错的勇气和探索的自由。在本项目中，撤销/重做功能的实现深度依赖于前述的混合数据模型设计，即结合了YAML快照和JSON Patch的策略。这种设计不仅在技术上可行，而且在用户体验层面带来了显著的提升，实现了即时响应与可靠历史回溯的平衡。

撤销功能的实现主要依赖于JSON Patch的增量记录机制。当用户执行任何一次可撤销的编辑操作时——无论是修改一个配置项、增删一个函数，还是调整一个变量的值——系统都会立即生成一个相应的JSON Patch对象，并将其存入一个操作堆栈（通常称为“撤销堆栈”）。这个堆栈遵循后进先出的原则，直观地反映了用户操作的历史顺序。当用户触发“撤销”命令时，系统只需从堆栈顶取出最近的一个Patch，计算其逆向Patch（inverse patch），然后将这个逆向Patch应用于当前状态，即可完成一步撤销。随后，这个正向的Patch会被移至“重做堆栈”。由于JSON Patch的每次变更都非常轻量，记录和计算逆向操作的开销极低，因此整个过程几乎是瞬时完成的，为用户提供了极其流畅和即时的反馈。这种机制支持“无限撤销”，即用户可以连续撤销多次操作，直到他们满意为止，这对于复杂的、多步骤的创作过程尤为重要。一些高级的实现甚至会开始将JSON Patch作为一等公民，增量地生成变更和逆向变更，从而在操作发生的第一时间就准备好撤销的能力 ¹。

然而，仅仅依赖JSON Patch进行撤销是不够的。想象一下，用户在一个大型项目中进行了数百次编辑，此时触发“撤销”功能，系统需要回溯并反向应用上百个Patch，这不仅计算量巨大，而且容易出错。更严重的是，如果在撤销过程中发生了程序崩溃或网络中断，那么这个长长的Patch序列就会丢失，用户的所有操作历史都将付之一炬。这就是YAML快照发挥关键作用的地方。系统会定期或在关键节点（如完成一个生成阶段、成功保存）创建一个完整的YAML快照。这个快照就像一个稳固的锚点，为撤销历史提供了一个可靠的起点。当用户需要进行大幅度的回退，或者在长时间的会话后想回到一个已知的良好状态时，系统可以直接加载最近的YAML快照，从而跳过漫长而复杂的逐条反向应用过程。这种“快照+补丁”的组合，为撤销/重做功能提供了双保险：日常的、小范围的撤销由高效的JSON Patch负责，而长期的、大规模的状态回溯则由稳健的YAML快照保障。

重做功能的实现逻辑与撤销类似，但方向相反。它主要依赖于那个存放被弹出的撤销操作的“重做堆栈”。当用户撤销了若干步之后，他们可以随时触发“重做”命令，系统会从重做堆栈中取出对应的操作（即最初的JSON Patch），将其重新应用于当前状态。重做堆栈的内容来源于用户成功的撤销操作。一旦一个操作被重做，它会从重做堆栈回到撤销堆栈，以便用户可以再次撤销它。这种双向的、基于堆栈的机制，构成了标准的撤销/重做历史管理模型。值得注意的是，YAML快照的存在也影响了重做流程。当用户加载一个新的快照后，重做堆栈通常会被清空，因为从一个全新的状态出发，之前的重做历史已经失去了意义。

综合来看，这种数据建模方式对用户体验的积极影响是全方位的。首先是降低心理门槛，即时的撤销反馈让用户敢于尝试各种可能性，不必担心自己的操作会带来不可挽回的后果，这对于一个旨在辅助用户进行创造性活动的平台来说至关重要。其次是提升操作效率，用户可以快速纠正笔误或无效操作，而无需手动重新输入或查找。再次是增强安全感和掌控感，知道无论发生什么，都能通过历史记录回到一个确定的、良好的工作状态，这种“容错”能力极大地增强了用户对系统的信任。最后，它与双向追溯功能形成了完美的互补。当用户在代码层面撤销或重做时，界面能够智能地联动更新，高亮显示受影响的规划节点，反之亦然。这种深层次的、语义层面的同步，使得用户在整个“意图-规划-实现”的价值链中拥有了前所未有的导航能力，真正实现了对生成过程的完全掌控。

模型路由策略：DeepSeek-v4-pro与DeepSeek-v4-flash的分工逻辑与性能权衡

项目中同时集成DeepSeek-v4-pro与DeepSeek-v4-flash两种模型，其根本目的在于通过智能的路由策略，在满足服务质量的前提下，最大化经济效益并优化用户体验。这是一种典型的自适应大语言模型路由问题，其核心是在成本、延迟和输出质量这三个相互制约的因素之间找到最佳平衡点 [13](#) [15](#)。路由策略的设计是项目智慧大脑的体现，决定了不同AI任务如何被高效、经济地分配给最适合的“工具”。

首先，必须清晰界定这两种模型的技术特性和能力边界。DeepSeek V4 Pro是一款专为复杂推理和高精度任务设计的旗舰模型 [58](#)。它采用了先进的混合专家架构，拥有高达1.6万亿的总参数量，但在处理每个令牌时仅激活约490亿个参数 [57](#)。这种设计使其在长上下文理解、世界知识掌握、复杂逻辑推理和解决需要创造性的代码重构等问题上表现出色 [57](#) [60](#)。相比之下，DeepSeek V4 Flash则是一款面向速度和成本优化的模型 [8](#)。它同样基于MoE架构，但规模更小，总参数量为284B，激活参数为13B [57](#)。它的核心优势在于极高的推理速度、超低的延迟和极具竞争力的价格。根据公开的API定价信息，V4-Pro的费用显著高于V4-Flash，例如，处理100万tokens的请求，V4-Pro的成本约为V4-Flash的10倍以上 [9](#)。这种巨大的性能和成本差异，为实施精细化的路由策略提供了坚实的基础。

基于上述特性，一个合理的分工逻辑和静态路由策略可以被制定出来。这种策略的核心思想是“按需分配”，即将复杂的、需要深度思考的任务交给V4-Pro，而将重复性的、模式化的、对延迟敏感的任务交给V4-Flash。

以下表格详细阐述了两模型的分工逻辑：

任务类型	推荐模型	设计依据与理由
顶层规划与意图理解	DeepSeek V4 Pro	需要深度理解用户模糊需求，进行抽象和高层设计，要求最强的推理和规划能力 37 39。
复杂代码重构与优化	DeepSeek V4 Pro	需要全局视野审视代码，发现潜在问题并进行创造性修改，对模型的理解力和创造力要求高 58。
长上下文代码库分析	DeepSeek V4 Pro	分析跨越数万token的代码库或文档，需要百万级别上下文窗口和强大的信息整合能力 57。
代码生成与实现填充	DeepSeek V4 Flash	任务本身是模式化的，Flash的成本效益更高，且其推理速度能满足快速迭代的需求 8 59。
单元测试生成	DeepSeek V4 Flash	编写遵循特定框架的测试代码是重复性工作，Flash足以胜任且能大幅降低成本 17。
代码补全与实时建议	DeepSeek V4 Flash	对延迟要求极高，Flash的高速推理是理想选择，能提供无缝的编码辅助体验 43。
简单查询与信息检索	DeepSeek V4 Flash	回答关于已有代码或文档的直接问题，属于低复杂度任务，使用低成本模型即可满足 8。

为了进一步提升效率和灵活性，可以引入一个动态自适应的路由器模块。这个路由器超越了简单的静态规则，能够根据实时反馈和上下文信息做出更精细化的决策。例如，它可以采用上下文感知路由，在代码生成阶段，如果V4-Flash生成的代码在初步验证中失败率持续偏高，路由器可以自动介入，降级或升级模型，甚至触发重试机制，或者通知V4-Pro接管处理。另一种策略是预算约束下的路由，系统可以为单次AI调用设定一个token消耗的预算上限。路由器会优先选择能满足质量要求且成本最低的模型；只有当低成本模型无法达到预期效果时，才会启用更高成本的模型 15。更前沿的方法是借鉴强化学习中的多臂老虎机理论，即基于Bandit的路由。路由器可以不断地尝试不同的模型组合，并根据历史的成功率、成本和耗时数据，动态地学习和调整路由策略，以实现长期的最优成本效益 54 56。

然而，实施复杂的路由策略也伴随着显著的工程取舍。首要的权衡是延迟、成本与质量的三角关系。V4-Pro提供了最高的质量，但代价是更高的延迟和成本。V4-Flash则恰好相反。路由策略的本质就是在三者之间寻找最佳平衡点。例如，对于一个交互式编程助手，降低延迟可能是首要目标，此时即使是牺牲少量质量也在所不惜。其次是实现复杂度与收益的权衡。静态路由相对简单，易于实现和维护。而动态自适应路由需要构建额外的监控、反馈和决策模块，这会显著增加系统的工程复杂度和运维成本 16。最后是一致性与灵活性的权衡。过于动态的路由可能导致结果不稳定，同一个任务在不同时间请求可能会得到略有差异的结果，这对某些严肃的应用场景（如金融交易逻辑生成）可能是不可接受的。因此，需要在追求极致优化的灵活性和确保结果确定性的稳定性之间找到一个微妙的平衡点。

综合分析与设计洞察

通过对项目在核心处理流程、数据模型设计及模型路由策略三个维度的深入探讨，一幅高度集成、智能化且注重用户体验的系统实现蓝图已然清晰。这些设计并非孤立存在，而是相互关联、彼此支撑，共同构成了项目的技术核心竞争力。本节将对前述分析进行综合提炼，形成对项目整体设计思路的深刻洞察。

首先，在宏观流程层面，项目采用的三阶段脚本生成管道不仅是任务执行的组织形式，更是对AI模型能力的一种精准映射。该管道将复杂的编码任务分解为“意图理解与规划”、“代码生成与验证”、“优化与重构”三个专业化阶段^{17 21}。这种分层流水线的思想，有效降低了单个阶段的认知负荷，提升了各阶段产出的专业水准。与此相呼应的是模型路由策略，它为此管道中的每个阶段配备了最合适的“专家”。DeepSeek V4 Pro如同一位经验丰富的“首席架构师”，被委以重任，专注于需要深度思考的第一阶段规划和第三阶段的复杂重构，确保顶层设计的正确性和最终产品的高质量^{57 58}。而DeepSeek V4 Flash则扮演着高效“资深工程师”的角色，承担起第二阶段具体的代码实现和测试生成等重复性、模式化且对成本敏感的任务^{8 59}。这种明确的分工逻辑，是实现项目在保证输出质量的同时，兼顾经济效益的关键所在，它体现了将复杂问题分解并匹配最合适工具的工程智慧。

其次，在微观数据层面，项目所采纳的结合了YAML快照和JSON Patch的混合数据模型，是实现流畅交互和强大追溯能力的技术基石。JSON Patch的增量记录机制为用户提供了一个即时、平滑的撤销体验，鼓励探索和创新；而YAML快照则作为可靠的历史锚点，确保了在发生严重错误后仍能安全回溯。这种“快慢结合”的存储策略，巧妙地平衡了存储效率与状态恢复的可靠性。尤为突出的是，该数据模型设计为实现SRS文档中强调的“双向追溯”功能提供了原生支持。通过在数据对象中嵌入source_ref元数据，系统能够建立从代码实现到其最初规划意图的清晰链接⁴⁷。这种链接关系使得用户可以在“意图-规划-实现”的不同抽象层次间自由穿梭，极大地提升了对生成结果的理解、调试和修改效率。它将用户从被动的代码使用者转变为主动的、可追溯的探索者，这种设计哲学深刻地影响了产品的专业性和可用性。

最后，模型路由策略是整个系统智慧的大脑，它贯穿于核心处理流程的始终。无论是基于任务性质的静态路由规则，还是更复杂的动态自适应策略，其根本目标都是为了让正确的工具（模型）在正确的时间做正确的事^{12 13}。这不仅关乎直接的API成本节约和响应延迟的优化，更直接影响到最终交付成果的质量和用户的满意度。通过在成本、性能与准确性之间进行审慎的权衡，项目能够在激烈的市场竞争中，提供一个既经济实惠又高质量的人工智能辅助开发解决方案。例如，虽然V4-Pro在百万Token上下文下的推理成本相比前代模型反而降低了约73%，但这并不意味着可以无限制使用，智能的路由仍然是必要的

综上所述，该项目的设计充分体现了现代AI应用开发的先进趋势。它通过分层流水线提升任务执行的专业性，通过精密的数据模型增强交互的流畅性和可追溯性，通过智能的调度策略优化服务的经济性。从宏观的Agentic Pipeline到微观的JSON Patch，再到顶层的路由决策，整个系统展现出高度的一致性和协同性。这份深度研究报告基于所提供的材料，对这些关键设计维度进行了详尽的解构与分析，揭示了其背后蕴含的深刻技术洞见与工程考量。

参考文献

1. Rethinking Undo/Redo - Why We Need Travels - DEV Community <https://dev.to/unadlib/rethinking-undoredo-why-we-need-travels-2lcc>
2. Undo redo for a huge object - javascript - Stack Overflow <https://stackoverflow.com/questions/58324932/undo-redo-for-a-huge-object>
3. DeepSeek-V3-Base - 模型详情页 <https://modelscope.cn/models/deepseek-ai/DeepSeek-V3-Base>
4. DeepSeek-V3 Technical Report - arXiv <https://arxiv.org/html/2412.19437v1>
5. DeepSeek | 深度求索 <https://www.deepseek.com/>
6. Schema-Grounded State Mutation for Reliable and Auditable LLM ... <https://arxiv.org/html/2605.29313v1>
7. RZ-NAS: Enhancing LLM-guided Neural Architecture Search via... <https://openreview.net/forum?id=9UEXqPH078>
8. DeepSeek V4 Preview Release <https://api-docs.deepseek.com/news/news260424>
9. Filip Nowak's Post - LinkedIn https://www.linkedin.com/posts/fibleep_deepseek-v4-dropped-this-morning-and-i-spent-activity-7453351491131826176-w9kh
10. DeepSeek V4 vs GPT-5.5, Claude Opus 4.7, Gemini 3.1 Pro ... https://www.linkedin.com/posts/martin-ciupa-76418b17_here-is-the-detailed-comparison-with-sources-activity-7462142255936647168-iKHL
11. ReLU vs Leaky ReLU: Activation Functions Explained - LinkedIn https://www.linkedin.com/posts/tom-yeh_aibyhahd-activity-7455675184440811521-FauU
12. BEST-Route: Adaptive LLM Routing with Test-Time Optimal Compute <https://arxiv.org/html/2506.22716v1>
13. [PDF] Adaptive Model and Strategy Routing for Cost-Efficient LLM Services http://home.ustc.edu.cn/~sa517494/files/www25_zhihong.pdf

14. BEST-Route: Adaptive LLM Routing with Test-Time Optimal Compute <https://openreview.net/forum?id=tFBibCVXkG>
15. [PDF] Adaptive LLM Routing Under Budget Constraints - ACL Anthology <https://aclanthology.org/2025.findings-emnlp.1301.pdf>
16. Adaptive LLM Routing under Budget Constraints | Yesudason Paulraj https://www.linkedin.com/posts/ypaulraj_adaptive-llm-routing-under-budget-constraints-activity-7370602675186515968-NPc5
17. Debunking Anti-AI Code Generation Myths | Bryan Finster posted on ... https://www.linkedin.com/posts/bryan-finster_so-the-repetitive-anti-ai-argument-has-moved-activity-7433133289462353920-weZ-
18. Reversa: A Reverse Documentation Engineering Framework ... - arXiv <https://arxiv.org/html/2605.18684v1>
19. How to write a good spec for AI agents - Addy Osmani <https://addyosmani.com/blog/good-spec/>
20. Lets play a game - heres your agentic pipeline controller production ... <https://community.openai.com/t/lets-play-a-game-heres-your-agentic-pipeline-controller-production-grade/1271056>
21. On Simulation-Guided LLM-based Code Generation for Safe ... <https://dl.acm.org/doi/full/10.1145/3756681.3756987>
22. Helping Users Update Intent Specifications for AI Memory at Scale <https://dl.acm.org/doi/full/10.1145/3746059.3747778>
23. Fundamentals and Practical Implications of Agentic AI - arXiv <https://arxiv.org/html/2505.19443v1>
24. Cost-Aware Query Routing in RAG: Empirical Analysis of Retrieval ... <https://arxiv.org/html/2606.02581v1>
25. [PDF] DeepSeek-V3 Technical Report - arXiv <https://arxiv.org/pdf/2412.19437>
26. [PDF] DeepSeek V3.2 Technical Documentation <https://fe-static.deepseek.com/chat/transparency/deepseek-v3.2-model-card-0414-EN.pdf>
27. [PDF] DeepSeek V4 Technical Documentation <https://fe-static.deepseek.com/chat/transparency/deepseek-V4-model-card-EN.pdf>
28. [PDF] DeepSeek-V3.2: Pushing the Frontier of Open Large Language ... <https://modelscope.cn/models/deepseek-ai/DeepSeek-V3.2/resolve/master/assets/paper.pdf>
29. DeepSeek API Docs: Your First API Call <https://api-docs.deepseek.com/>
30. Anthropic API - DeepSeek API Docs https://api-docs.deepseek.com/guides/anthropic_api
31. DeepSeek-V3.2 Release <https://api-docs.deepseek.com/news/news251201>

32. The Temperature Parameter - DeepSeek API Docs https://api-docs.deepseek.com/quick_start/parameter_settings
33. [PDF] AWS Transform - User Guide <https://docs.aws.amazon.com/pdfs/transform/latest/userguide/AWSTransformUserGuide.pdf>
34. bing.txt - FTP Directory Listing <ftp://ftp.cs.princeton.edu/pub/cs226/autocomplete/bing.txt>
35. DeepSeek V4 Whitepaper Released with AI Concepts Explained https://www.linkedin.com/posts/shao-hang-he_deepseek-youtube-artificialintelligence-activity-7457457631058829312-i1JL
36. Integrate with Claude Code | DeepSeek API Docs https://api-docs.deepseek.com/quick_start/agent_integrations/claude_code
37. Thinking Mode - DeepSeek API Docs https://api-docs.deepseek.com/guides/thinking_mode
38. Rate Limit & Isolation - DeepSeek API Docs https://api-docs.deepseek.com/quick_start/rate_limit
39. Create Chat Completion - DeepSeek API Docs <https://api-docs.deepseek.com/api/create-chat-completion>
40. Token & Token Usage - DeepSeek API Docs https://api-docs.deepseek.com/quick_start/token_usage
41. Computer Science - arXiv <https://www.arxiv.org/list/cs/new?skip=150&show=1000>
42. DeepSeek Terms of Use <https://cdn.deepseek.com/policies/en-US/deepseek-terms-of-use.html>
43. Introducing DeepSeek-V3 <https://api-docs.deepseek.com/news/news1226>
44. DeepSeek-R1 Release <https://api-docs.deepseek.com/news/news250120>
45. I Tested DeepSeek V4 Flash and GPT-4o Side by Side <https://dev.to/eagerspark/i-tested-deepseek-v4-flash-and-gpt-4o-side-by-side-heres-the-p99-latency-truth-3jd1>
46. DeepSeek-Inspired Exploration of RL-based LLMs and Synergy with ... <https://arxiv.org/abs/2503.09956>
47. DeepSeek V4 Model Delay Confirmed | Paul S. Triolo posted on the ... https://www.linkedin.com/posts/paul-s-triolo-26825925_aistackdecrypted-paul-triolo-substack-activity-7446924567811510273-KJNE
48. 聊聊DeepSeek V4：这份技术报告里藏着的那些"小字"才是重头戏- 知乎 <https://zhuanlan.zhihu.com/p/2031168317549966085>
49. DeepSeek is back among the leading open weights models with V4 ... <https://www.linkedin.com/pulse/deepseek-back-among-leading-open-weights-models-v4-gaktc>

50. Empirical, Attention-Based and Mechanistic Insights into Distilled ... <https://aclanthology.org/2025.emnlp-main.198/>
51. [PDF] ORI: O Routing Intelligence - arXiv <https://arxiv.org/pdf/2502.10051>
52. ACL ARR 2026 January Submissions - OpenReview <https://openreview.net/submissions?page=123&venue=aclweb.org%2FACL%2FARR%2F2026%2FJanuary>
53. Arxiv今日论文| 2026-05-21 - 闲记算法 http://lonepatient.top/2026/05/21/arxiv_papers_2026-05-21.html
54. Adaptive LLM Routing under Budget Constraints - ACL Anthology <https://aclanthology.org/2025.findings-emnlp.1301/>
55. 仅5000行代码改写规则！ V4 架构专治推理I/O瓶颈，性能暴增187% <https://www.infoq.cn/article/G8pD8clar51Ssg6GwXmB>
56. [2508.21141] Adaptive LLM Routing under Budget Constraints - arXiv <https://arxiv.org/abs/2508.21141>
57. 过年了？ DeepSeek-V4 技术报告快速解读 - 知乎专栏 <https://zhuanlan.zhihu.com/p/2030970565092177542>
58. DeepSeek-V4 技术报告详细解读 - 知乎专栏 <https://zhuanlan.zhihu.com/p/2031116021499679430>
59. Deepseek V4 Flash! 是否真的能打？ 实测报告！ <https://www.bilibili.com/video/BV1ZY05BLELZ/>
60. 为什么说DeepSeek V4 不一样？ 看完你就懂了 - 电子工程专辑 <https://www.eet-china.com/mp/a490841.html>